

and slow response times. Other services are slow because they're waiting on it.

They jump to the logs for the recommendations service. A flood of timeout errors talking to the model server. They go to the trace view, pick a recent slow request, and see the entire flow. Frontend to API gateway to user service to recommendations service to model server. The model server is taking eight seconds to respond, when it should take under two hundred milliseconds.

A quick check of recent deployments. Yes, the model server got a new version pushed two hours ago. The engineer rolls back. Latency recovers within ninety seconds. They write a quick note in the incident channel, file a ticket for the postmortem, and go back to dinner.

Total time from alert to resolution, about eight minutes. Without observability tooling, the same incident might have taken hours, lots of guessing, and probably a few wrong fixes along the way.

Metrics are the heartbeat of the city

Of the three pillars, metrics are usually where teams start. They're cheap to collect, cheap to store, and cheap to query. They give you the broad shape of what's happening.

Prometheus is the open source heavyweight in this space. It pulls metrics from every service and stores them in a time-series database. You can write queries to slice and aggregate the data in basically any way you can imagine.

The catch with metrics is that they tell you what, but not always why. They tell you that the error rate jumped. They don't tell you which user, which request, which line of code. For that, you need to drop down to logs and traces.

Logs are the diary, but a chaotic one

Logs are the oldest form of telling-the-computer-to-tell-you-what-it-did. Every application is designed to emit them. Every server collects them.

The challenge has never been getting logs. The challenge has always been finding the one log line that matters in a sea of millions.

The modern approach is structured logging. Instead of printing free-form strings, applications emit logs as JSON or similar formats with consistent fields. That way you can query them.

Tools like Loki, Elastic, and OpenSearch handle log storage at scale. They index the data so queries are fast. They typically have a dashboard interface, like Kibana for Elastic or Grafana for Loki, where you can search, filter, and visualise.

Traces are the map of how requests really travel

Traces are the youngest of the three pillars and the one most people understand least, even though they're often the most useful when things go wrong.

A trace follows a single request as it moves through your system. The browser sends a request to the gateway. The gateway calls the user service. The user service calls the database and the recommendations service. The recommendations service calls the model server. Each of these hops gets recorded as a span. All the spans for one request get linked together as a trace.

When you look at a trace in a tool like Jaeger or Tempo, you see a waterfall view. Each span is a horizontal bar. The width tells you how long it took. You can see exactly where time was spent. If the recommendations service is taking eight seconds, you can see whether it was the model server call, the database query, or something else entirely.

The work of getting good traces is mostly about instrumentation. Each service needs to know how to receive an incoming trace context, pass it along to downstream calls, and emit its own spans. OpenTelemetry has emerged as the standard way to do this, and most modern languages and frameworks have OpenTelemetry libraries that handle most of the work automatically.

Where SLOs come in

There's one more concept that needs to be in this conversation -- Service Level Objectives (SLOs).

The basic idea is simple. You define what acceptable performance looks like for each service in terms users would actually care about: Page load time is under three seconds for 99 percent of requests. Search results returned in under 500 milliseconds for 95 percent of requests. The checkout API is available 99.9 percent of the time.

Then you measure whether you're meeting those targets. If you are, life is good. If you're not, you have an error budget that's getting depleted, and your team needs to slow down feature work and focus on reliability.

SLOs are powerful because they shift the conversation from intuition to data. Instead of arguing about whether the site is "slow" or whether the service is "stable," you point at a number. The SLO says 99.5 percent. We're at 98.9 percent this month. Here's what's been eating the budget.

Most modern observability platforms have built-in SLO tracking. Grafana has SLO dashboards. Sloth is an open source tool specifically for managing SLOs against Prometheus metrics. The setup work is real but pays off enormously in clearer prioritisation conversations.

The role of automation

Observability alone doesn't keep a data centre running. People do, with the help of automation. A modern operations setup typically includes a lot of automated responses to common problems.

If a host goes unhealthy, it gets drained and replaced automatically. If a service is using too much memory, it gets restarted. If traffic in one region spikes, additional capacity gets provisioned. If a deployment causes elevated error rates, it gets rolled back without human intervention.

None of this automation works without good signals. The automation needs to know what healthy looks